JS Essential Methods Guide

JavaScript Master Reference Guide (Essential Built-in Methods)

This guide focuses on the **20% of JavaScript methods that solve 80% of real-world problems**.

---

1. Array Transformation & Iteration

A. `map()`

Syntax

```JavaScript
array.map((item, index, array) => {})
```

Definition

Creates a **new array** by transforming every element.

Example

```JavaScript
const prices = [100, 200, 300];

const gstPrices = prices.map(price => price * 1.18);

console.log(gstPrices);
// [118, 236, 354]
```

Gotcha

- Does NOT modify original array.
- Must return a value.

B. `forEach()`

Syntax

```JavaScript
```

```
array.forEach((item, index) => {})
```

Definition

Executes a function for each element and returns `undefined`.

Example

JavaScript

```
const users = ["Nilesh", "Rahul"];

users.forEach(user => {
    console.log(`Welcome ${user}`);
});
```

Gotcha

Cannot use `break`, `continue`, or return a new array.

C. `filter()`

Syntax

JavaScript

```
array.filter((item) => condition)
```

Definition

Returns a new array containing only elements that pass the condition.

Example

JavaScript

```
const marks = [45, 80, 30, 90];

const passed = marks.filter(mark => mark >= 40);

console.log(passed);
// [45,80,90]
```

Gotcha

Returns an array even if only one item matches.

D. `reduce()`

Syntax

```JavaScript
array.reduce((accumulator, current) => {}, initialValue)
```

Definition

Reduces an array into a single value.

Example

```JavaScript
const cart = [100, 200, 300];

const total = cart.reduce((sum, item) => sum + item, 0);

console.log(total);
// 600
```

Gotcha

Always provide an initial value.

E. `flatMap()`

Syntax

```JavaScript
array.flatMap(callback)
```

Definition

Maps and flattens one level simultaneously.

Example

```JavaScript
const words = ["hello world", "js"];

const result = words.flatMap(word => word.split(" "));

console.log(result);
// ['hello','world','js']
```

Gotcha

Only flattens one level.

F. `flat()`

Syntax

JavaScript

```
array.flat(depth)
```

Definition

Flattens nested arrays.

Example

JavaScript

```
const arr = [1,[2,[3]]];

console.log(arr.flat(2));
// [1,2,3]
```

Gotcha

Deep nesting may affect performance.

G. `sort()` (Mutates)

Syntax

JavaScript

```
array.sort(compareFn)
```

Definition

Sorts array in place.

Example

JavaScript

```
const nums = [10,2,5];

nums.sort((a,b)=>a-b);
```

```
console.log(nums);
// [2,5,10]
```

Gotcha

Without compare function:

JavaScript

```
[10,2,5].sort()
// [10,2,5]
```

Lexicographical sorting happens.

H. `reverse()` (Mutates)

Syntax

JavaScript

```
array.reverse()
```

Definition

Reverses original array.

Example

JavaScript

```
const arr = [1,2,3];

arr.reverse();

console.log(arr);
// [3,2,1]
```

Gotcha

Mutates original array.

---

2. Array Searching & Validation

A. `find()`

Syntax

```JavaScript
array.find(callback)
```

Definition

Returns first matching element.

Example

```JavaScript
const users = [
  {id:1},
  {id:2}
];

const user = users.find(u => u.id === 2);

console.log(user);
```

Gotcha

Returns `undefined` if not found.

B. `findIndex()`

Syntax

```JavaScript
array.findIndex(callback)
```

Definition

Returns index of first match.

Example

```JavaScript
const arr = [10,20,30];

console.log(
    arr.findIndex(x => x === 20)
);
//1
```

Gotcha

Returns `-1` if not found.

C. `includes()`

Syntax

JavaScript

```
array.includes(value)
```

Definition

Checks whether value exists.

Example

JavaScript

```
const skills = ["HTML","CSS","JS"];

console.log(
    skills.includes("JS")
);
// true
```

Gotcha

Case-sensitive.

D. `some()`

Syntax

JavaScript

```
array.some(callback)
```

Definition

Returns true if at least one item matches.

Example

JavaScript

```
const nums = [1,2,3];

console.log(
 nums.some(num => num > 2)
);
// true
```

Gotcha

Stops at first match.

E. `every()`

Syntax

JavaScript

```
array.every(callback)
```

Definition

Returns true if all elements satisfy condition.

Example

JavaScript

```
const ages = [20,25,30];

console.log(
 ages.every(age => age >=18)
);
// true
```

Gotcha

Returns false immediately on first failure.

---

3. Object Manipulation & Serialization

A. `Object.keys()`

Syntax

JavaScript

```
Object.keys(obj)
```

Definition

Returns array of property names.

Example

JavaScript

```
const user = {
  name:"Nilesh",
  age:20
};

console.log(
 Object.keys(user)
);
// ['name','age']
```

Gotcha

Only enumerable properties.

B. `Object.values()`

Syntax

JavaScript

```
Object.values(obj)
```

Definition

Returns array of values.

Example

JavaScript

```
console.log(
 Object.values({
   a:1,
   b:2
 })
);
// [1,2]
```

C. `Object.entries()`

Syntax

JavaScript

```
Object.entries(obj)
```

Definition

Returns key-value pairs.

Example

JavaScript

```
const obj = {
  name:"Nilesh"
};

console.log(
 Object.entries(obj)
);
// [['name','Nilesh']]
```

D. `Object.assign()`

Syntax

JavaScript

```
Object.assign(target, source)
```

Definition

Copies properties into target object.

Example

JavaScript

```
const user = {
  name:"Nilesh"
};

const updated = Object.assign(
 {},
 user,
 {role:"Admin"}
```

```
);

console.log(updated);
```

Gotcha

Shallow copy only.

E. Spread Operator

Syntax

JavaScript

```
{...object}
```

Definition

Creates a shallow copy.

Example

JavaScript

```
const user = {
  name:"Nilesh"
};

const copy = {
  ...user,
  role:"Developer"
};
```

Gotcha

Nested objects remain referenced.

---

4. String & Number Formatting

A. `split()`

Syntax

JavaScript

```
string.split(separator)
```

Definition

Converts string into array.

Example

JavaScript

```
const skills = "HTML,CSS,JS";

console.log(
 skills.split(",")
);
```

B. `join()`

Syntax

JavaScript

```
array.join(separator)
```

Definition

Converts array into string.

Example

JavaScript

```
const arr = ["HTML","CSS"];

console.log(
 arr.join(" | ")
);
```

C. `trim()`

Syntax

JavaScript

```
string.trim()
```

Definition

Removes whitespace from both ends.

Example

JavaScript

```
const name = " Nilesh ";

console.log(
 name.trim()
);
```

D. `replace()`

Syntax

JavaScript

```
string.replace(old,new)
```

Definition

Replaces first match.

Example

JavaScript

```
const msg = "Hello Java";

console.log(
 msg.replace("Java","JavaScript")
);
```

E. `toFixed()`

Syntax

JavaScript

```
number.toFixed(digits)
```

Definition

Returns formatted decimal string.

Example

JavaScript

```
const price = 99.999;

console.log(
 price.toFixed(2)
);
// 100.00
```

Gotcha

Returns string, not number.

F. `Number()`

Syntax

JavaScript

```
Number(value)
```

Definition

Converts value to number.

Example

JavaScript

```
const age = Number("25");

console.log(age);
```

G. `parseInt()`

Syntax

JavaScript

```
parseInt(string, radix)
```

Definition

Parses integer.

Example

```JavaScript
parseInt("100px");
//100
```

H. `parseFloat()`

Syntax

```JavaScript
parseFloat(value)
```

Definition

Parses decimal number.

Example

```JavaScript
parseFloat("12.34px");
//12.34
```

---

5. Modern Async Control Flow

A. `async`

Syntax

```JavaScript
async function getData(){}
```

Definition

Makes function return a Promise.

Example

```JavaScript
```

```
async function hello(){
    return "Hi";
}
```

B. `await`

Syntax

JavaScript

```
await promise
```

Definition

Waits for Promise resolution.

Example

JavaScript

```
async function getUser(){

 const res = await fetch("/api/user");

 const data = await res.json();

 return data;
}
```

Gotcha

Only inside async functions.

C. `fetch()`

Syntax

JavaScript

```
fetch(url, options)
```

Definition

Returns Promise for HTTP request.

Example

```JavaScript
const users = await fetch(
  "https://jsonplaceholder.typicode.com/users"
);

const data = await users.json();
```

Gotcha

Fetch doesn't reject for 404/500.

```JavaScript
if(!response.ok){
   throw new Error("Failed");
}
```

D. `Promise.all()`

Syntax

```JavaScript
Promise.all(arrayOfPromises)
```

Definition

Runs multiple promises in parallel.

Example

```JavaScript
const [users, posts] =
await Promise.all([
 fetch("/users"),
 fetch("/posts")
]);
```

Gotcha

One rejection fails all.

E. `Promise.allSettled()`

Syntax

```JavaScript
Promise.allSettled(promises)
```

Definition

Waits for all promises regardless of success.

Example

```JavaScript
const result =
await Promise.allSettled([
 promise1,
 promise2
]);
```

Gotcha

Useful when partial failures are acceptable.

---

6. Timers & Event Loop

A. `setTimeout()`

Syntax

```JavaScript
setTimeout(callback, delay)
```

Definition

Runs function once after delay.

Example

```JavaScript
setTimeout(()=>{
 console.log("Hello");
},2000);
```

B. `clearTimeout()`

Syntax

```JavaScript
clearTimeout(id)
```

Example

```JavaScript
const timer =
setTimeout(()=>{},5000);

clearTimeout(timer);
```

C. `setInterval()`

Syntax

```JavaScript
setInterval(callback,time)
```

Definition

Runs repeatedly.

Example

```JavaScript
setInterval(()=>{
 console.log("Tick");
},1000);
```

Gotcha

Can cause memory leaks.

D. `clearInterval()`

Syntax

```JavaScript
clearInterval(id)
```

Example

```JavaScript
const id =
setInterval(()=>{},1000);

clearInterval(id);
```

---

7. JSON Data Handling

A. `JSON.stringify()`

Syntax

```JavaScript
JSON.stringify(value)
```

Definition

Converts JavaScript object into JSON string.

Example

```JavaScript
const user = {
 name:"Nilesh"
};

const json =
JSON.stringify(user);

console.log(json);
```

Output:

```JavaScript
'{"name":"Nilesh"}'
```

Gotcha

Functions and undefined are ignored.

B. `JSON.parse()`

Syntax

```JavaScript
JSON.parse(jsonString)
```

Definition

Converts JSON string into JavaScript object.

Example

```JavaScript
const data =
'{"name":"Nilesh"}';

const user =
JSON.parse(data);

console.log(user.name);
```

Gotcha

Invalid JSON throws error.

```JavaScript
try{
   JSON.parse(data);
}catch(err){
   console.log(err);
}
```

Top 15 JavaScript Methods You Should Master First

1. `map()`
2. `filter()`
3. `reduce()`
4. `find()`
5. `some()`
6. `every()`
7. `includes()`
8. `Object.keys()`
9. `Object.entries()`

10. `Object.values()`
11. `JSON.parse()`
12. `JSON.stringify()`
13. `fetch()`
14. `Promise.all()`
15. `async/await`

If you master these 15 methods deeply, you'll be able to build almost every MERN-stack project, admin dashboard, weather app, portfolio project, and real-world frontend application efficiently.

Sources